arXiv:2605.18177v2 [cs.CV] 13 Sep 2026

Token-Space Mask Prediction for Efficient Vision Transformer Segmentation

Calvin Galagain[1,2] Martyna Poreba[1] François Goulette[2]
[1]Université Paris-Saclay, CEA List [2]U2IS, ENSTA Paris, Institut Polytechnique de Paris
F-91120 Palaiseau, France
{calvin.galagain, martyna.poreba}@cea.fr francois.goulette@ensta.fr

Abstract

Query-based Vision Transformer segmentation models typically reconstruct dense spatial feature maps to predict masks, inheriting design patterns from convolutional architectures. We show that this explicit image-space reconstruction is not required. We introduce TokenMask, a token-space mask head that computes mask logits directly from query-token affinities and performs interpolation in logit space rather than feature space. This reformulation preserves the original linear scoring mechanism while simplifying the computational structure. Across diverse ViT backbones, datasets and segmentation tasks, TokenMask consistently improves efficiency over prior approaches by reducing computational and memory requirements while maintaining competitive accuracy, leading to tangible speedups on NVIDIA Jetson AGX Orin using TensorRT FP16 inference. Overall, TokenMask yields a simpler and more deployment-friendly design for embedded vision systems.

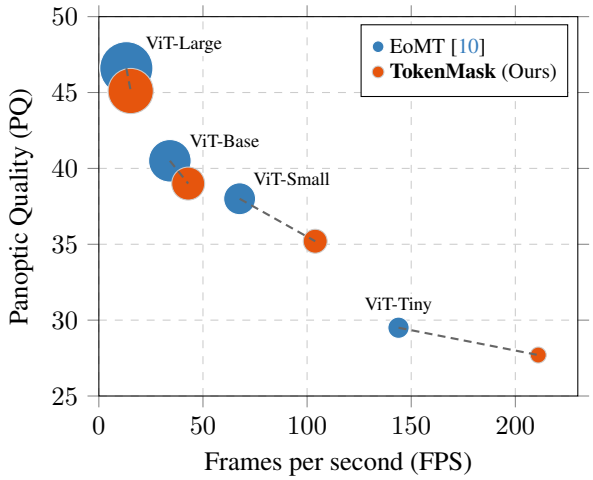

Figure 1. Efficiency-accuracy trade-off on ADE20K panoptic segmentation. Marker size is proportional to the number of GFLOPs. Dashed connectors show the transition from EoMT to TokenMask for each backbone size.

1. Introduction

Visual segmentation is a core problem in computer vision, aiming to assign semantic labels to every pixel in an image. It is typically studied under three settings: semantic, instance, and panoptic segmentation. These differ in their level of granularity, with panoptic segmentation unifying the former two by jointly modeling *stuff* and *thing* categories. These capabilities are critical for embedded vision applications such as robotics and intelligent systems, where accurate scene understanding must be achieved under strict real-time and computational constraints [7].

Recent advances in deep learning have significantly improved segmentation performance, notably through the introduction of Vision Transformers (ViTs) [6]. In this context, query-based approaches have emerged as a powerful paradigm [1], representing objects or regions as a set of learnable queries and enabling unified, global reasoning across different segmentation tasks. However, despite their conceptual appeal, existing query-based segmentation architectures remain computationally heavy and structurally complex, limiting their applicability to embedded platforms. Many existing query-based segmentation models rely on multi-stage decoding pipelines that combine dense pixel decoders with multi-layer transformer query decoders [2, 3]. While effective, this design couples token-based representations with explicit spatial decoding stages, introducing additional complexity that is not intrinsic to ViT-based segmentation. In particular, although these models operate on token sequences internally, mask prediction is still commonly performed in image space. Patch tokens are first reconstructed into dense spatial feature maps, often followed by feature upsampling to recover spatial resolution, before computing spatial interactions with segmentation queries. This design mirrors convolutional segmentation pipelines and reflects architectural conventions rather than a fundamental modeling requirement, leading to increased memory footprint and latency that are problematic for real-time embedded deployment.

This observation motivates a rethinking of mask prediction in query-based segmentation models. We move away from explicit spatial reconstruction and instead align the segmentation head with the token-based representation inherent to ViTs. From this perspective, we revisit the role of upsampling under resource constraints. We introduce TokenMask, a unified decoding head that can be seamlessly attached to a ViT backbone, and jointly supports instance, panoptic and semantic segmentation. As illustrated in Fig. 1, TokenMask preserves segmentation quality across ViT backbones while reducing computation by up to 50–55% compared to EoMT [10]. It also consistently improves throughput, with up to 45–50% higher FPS on Nvidia Jeston Orin.

Our contributions are threefold: **(1)** We reformulate mask prediction for ViT-based segmentation directly in token space, eliminating explicit spatial feature-map reconstruction while preserving the original linear scoring mechanism; **(2)** We analyze the impact of relocating upsampling from feature space to logit space and investigate regimes in which upsampling can be removed entirely; **(3)** We conduct a systematic evaluation across multiple backbones, datasets, and output strides, highlighting the computational and memory implications of mask-head design choices for embedded deployment.

2. Related Work

Semantic segmentation with ViTs has evolved primarily through the design of segmentation heads. Early approaches such as SETR [17, 18] rely on CNN-based upsampling heads to recover spatial predictions from patch tokens. In contrast, Segmenter [13] and StructToken [11] introduce fully Transformer-based heads, leveraging learnable class or structure tokens that interact with patch embeddings to directly produce segmentation masks. SegFormer [14] adopts a different strategy with a lightweight MLP head that efficiently fuses multi-scale features. More recently, SegViT [15] and SegViT2 [16] depart from explicit decoding by proposing an attention-to-mask mechanism, where class tokens directly generate segmentation masks from similarity maps.

Compared to dense ViT-based segmentation, query-based approaches predict a structured set of object and region masks rather than per-pixel labels. This formulation reduces heuristic post-processing, enables multi-task unification, and provides explicit control over complexity via the number of queries. It also promotes object-level reasoning and more efficient computation by focusing on a limited set of relevant regions instead of dense token-wise predictions. The first query-based segmentation approaches build upon DETR [1] by extending its set-based prediction framework with mask prediction heads, allowing each learned query to generate an object mask through interactions with dense image features. In these approaches, mask prediction is typically formulated in image space by projecting query embeddings onto spatial feature maps, tightly coupling query-based reasoning with dense spatial representations.

Building on this paradigm, MaskFormer [2] reformulates semantic segmentation as a mask classification problem, predicting a set of class-labeled masks using learnable queries. Mask2Former [3] extends query-based segmentation to a unified setting covering semantic, instance, and panoptic tasks by introducing a dense pixel decoder coupled with a multi-layer transformer query decoder for progressive query refinement. Mask prediction is achieved by associating each query with a learned mask representation that interacts with dense spatial features to produce segmentation outputs. Although this interaction is implemented in image space via dense feature maps, it effectively reduces to a linear similarity operation between query representations and spatially organized features. Mask2Former has become a reference architecture for query-based segmentation, and several subsequent works build upon it while targeting different objectives. OneFormer [9] extends the framework with task-conditioned queries to enable unified multi-task segmentation without architectural changes. EoMT [10] targets efficiency by simplifying the query-based segmentation pipeline. It removes several task-specific components commonly used in Mask2Former architecture, including the pixel decoder and the transformer query decoder, and repurposes the ViT encoder itself to jointly process image tokens and object queries. EoMT enables one-stage segmentation with significantly lower latency, while still relying on image-space mask prediction. Recently, LiPS [8] identifies a structural inefficiency in Mask2Former-like architectures, where the majority of computation is devoted to dense multi-scale feature processing rather than query-based reasoning. To address this, it preserves the masked transformer decoding paradigm while simplifying the upstream pipeline through selective feature routing and spatial compression, leading to a substantially improved accuracy-efficiency trade-off.

3. Methodology

Existing query-based segmentation architectures [3, 10], retain image-space mask heads inherited from dense decoders, requiring spatial reconstruction and feature upsampling. This design increases memory footprint and limits its scalability. To address this, we propose TokenMask, a lightweight token-space segmentation head for embedded ViTs that reduces memory and computational overhead while supporting unified semantic, instance, and panoptic segmentation.

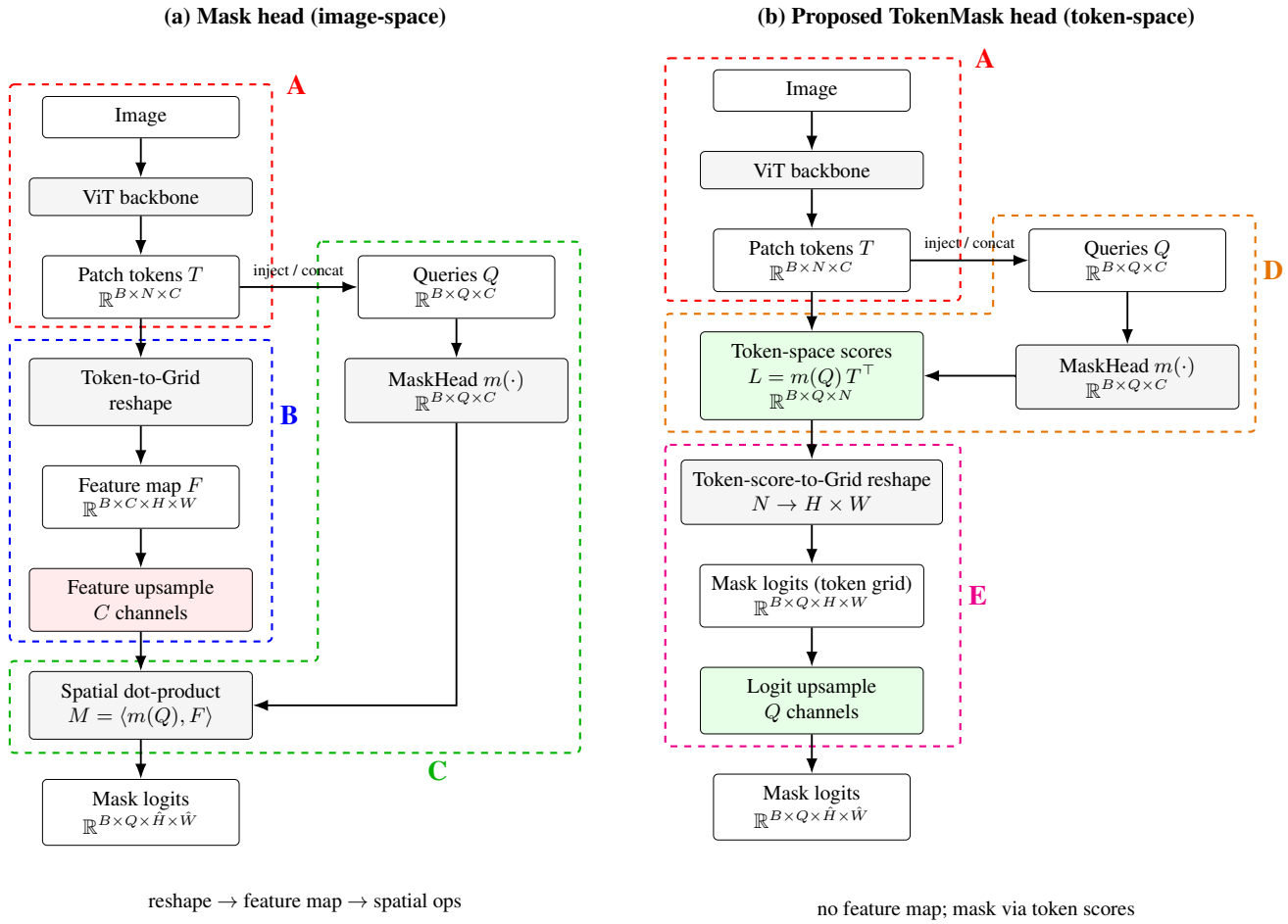

Figure 2. ViT backbone interfacing with Mask (image-space) and proposed TokenMask (token-space mask) heads: (a) Classical design reshapes patch tokens into an image-space feature map and computes masks via a dense spatial dot-product after upsampling C-channel features. (b) Proposed TokenMask head. Mask logits are computed directly in token space via $L = m(Q)T^\top$ and upsampled in logit space over Q channels, eliminating the need for explicit image-space feature maps. **A** corresponds to tokenization and ViT backbone processing that produce patch tokens. **B** denotes spatial reconstruction for mask prediction through token-to-grid reshaping and feature-space upsampling. **C** indicates query-based mask prediction via spatial dot-product in image space. **D** represents direct token-level scoring through query-token affinity computation. **E** corresponds to reshaping token-level scores to the spatial grid followed by logit-space upscaling.

3.1. Image-Space Mask Head

Tokenization and ViT backbone. Let the input image be $x \in \mathbb{R}^{B\times 3\times H_0\times W_0}$ where B denotes the batch size and H_0, W_0 the input spatial resolution. The image is partitioned into non-overlapping patches that are linearly projected into C-dimensional embeddings and processed by a ViT backbone. The backbone outputs a sequence of patch tokens:

$$T \in \mathbb{R}^{B\times N\times C}, \tag{1}$$

where $N = H_p \times W_p$ denotes the patch-grid resolution. Each token encodes visual content from a specific spatial region together with positional information, forming the core internal representation used by ViT-based segmentation models.

Spatial reconstruction for mask prediction. In most query-based segmentation architectures, mask prediction is performed in image space. As illustrated in Fig. 2 (a), the patch tokens T are first reshaped or rearranged into a dense spatial feature map:

$$F = \text{reshape}(T) \in \mathbb{R}^{B\times C\times H_p\times W_p}. \tag{2}$$

This reshape operation introduces no new information and merely restores an image-like tensor organization from the token sequence. The resulting feature map F is typically processed by a pixel decoder and optionally upsampled

along the spatial dimensions to recover resolution:

$$\tilde{F} \in \mathbb{R}^{B\times C\times \hat{H}\times \hat{W}}, \tag{3}$$

where $\hat{H}$ and $\hat{W}$ denote the upsampled spatial dimensions. Upsampling is applied per-channel; hence compute and memory scale linearly with the channel dimension C.

Query-based mask prediction. Query-based models maintain a fixed set of learnable queries:

$$Q \in \mathbb{R}^{B\times Q\times C}, \tag{4}$$

where Q denotes the number of queries. The queries are injected into a transformer decoder and refined through interactions with image features, yielding query representations used for mask prediction. Each query is projected by a mask head into a mask embedding, and segmentation masks are obtained via a spatial dot-product with the feature map F:

$$M = m(Q) \cdot F, \tag{5}$$

where $m(\cdot)$ denotes a learned linear projection. The resulting tensor:

$$M \in \mathbb{R}^{B\times Q\times H_p\times W_p} \tag{6}$$

corresponds to the mask logits for all queries, which are subsequently used for loss computation and final segmentation decoding. Although implemented as a spatial operation over dense feature maps, this mechanism fundamentally corresponds to a linear interaction between query embeddings and spatially organized token representations.

3.2. Token-Space Mask Head: TokenMask

Direct token-level scoring. As illustrated in Fig. 2 (b), we reformulate mask prediction directly in token space and remove the explicit reconstruction of spatial feature maps. Given the patch tokens extracted by the ViT backbone and the projected query embeddings from the mask head, we compute mask scores directly at the token level via a matrix multiplication:

$$L = m(Q)T^{\top}, \tag{7}$$

yielding:

$$L \in \mathbb{R}^{B\times Q\times N}. \tag{8}$$

Each entry:

$$L_{q,i} = \langle m(q), t_i \rangle \tag{9}$$

represents the affinity between query q and patch token i, and can be interpreted as a membership score indicating how strongly token i contributes to the mask predicted by query q. This operation is strictly equivalent to the spatial dot-product used in image-space mask heads (Fig. 2 (a)), but is performed directly on the token sequence without intermediate spatial reconstruction.

Reshaping and logit-space upsampling. The token-level scores are then reshaped back to the patch grid:

$$M_{\text{patch}} \in \mathbb{R}^{B\times Q\times H_p\times W_p}, \tag{10}$$

recovering a coarse spatial layout aligned with the patch resolution. To obtain the final mask predictions, interpolation is applied directly in logit space:

$$M \in \mathbb{R}^{B\times Q\times \hat{H}\times \hat{W}}. \tag{11}$$

Importantly, the proposed formulation does not modify the underlying linear scoring mechanism. The mask remains a projection between query embeddings and local representations.

3.3. Implications for Embedded Deployment

By design, TokenMask eliminates feature-space upsampling and operates exclusively on compact token-level representations. This choice substantially reduces memory access cost and intermediate activation storage, two dominant bottlenecks in embedded vision pipelines where off-chip memory access often outweighs arithmetic cost.

First, TokenMask avoids explicit spatial feature reconstruction, thereby preventing the formation of large intermediate feature maps whose size scales with both spatial resolution and channel dimensionality C, which can be large in practice (e.g., $C \in [384, 1024]$). In conventional pipelines, this reconstruction step reorganizes token sequences into dense, image-like tensors that must be materialized in memory, leading to increased memory footprint and substantial data movement. In contrast, TokenMask computes mask scores directly in token space through a matrix multiplication between projected queries and patch tokens. This computation maps naturally to batched general matrix-matrix multiplication (GEMM), a highly optimized primitive on modern hardware accelerators, including embedded GPUs, NPUs, and DSPs. Leveraging GEMM enables efficient use of vector units, high arithmetic intensity, and favorable data reuse within on-chip memory, leading to improved throughput and reduced energy per inference.

Moreover, relocating upsampling from feature space to logit space further improves efficiency, as interpolation is performed over Q channels rather than C, with typically $Q \ll C$. This decouples the computational cost of mask prediction from the backbone embedding dimension and yields a segmentation head whose complexity scales more favorably with model capacity and input resolution.

4. Experiments

We evaluate the proposed TokenMask head by analyzing its impact on both segmentation accuracy and computational efficiency. Unless stated otherwise, all models share identical initialization, query count, decoder depth, and training protocol; observed differences therefore stem from the

Table 1. Performance and efficiency comparison on ADE20K for panoptic and semantic segmentation. Panoptic results are reported at 640×640 input resolution, and semantic results at 512×512.

		Panoptic				Semantic			
Backbone	**Method**	**FPS A100** ↑	**FPS Orin** ↑	**GFLOPs** ↓	**PQ** ↑	**FPS A100** ↑	**FPS Orin** ↑	**GFLOPs** ↓	**mIoU** ↑
ResNet-50	Mask2Former	0.6	7.1	147.2	39.6	0.6	7.1	147.2	46.0
AFFormer [5]	LiPS	3.6	17.5	26.4	35.8	3.6	17.5	26.4	44.0
ViT-Tiny	Segmenter	-	-	-	-	147.4	69.0	12.8	40.0
	EoMT	189.0	143.9	20.8	29.5	192.4	248.0	13.6	40.0
	TokenMask (Ours)	202.9	211.0	9.7	27.7	211.0	363.6	6.4	39.1
ViT-Small	Segmenter	-	-	-	-	94.9	69.0	38.6	46.5
	EoMT	129.3	67.6	71.7	38.0	133.0	114.4	46.7	45.2
	TokenMask (Ours)	150.8	103.9	37.3	35.2	151.8	175.4	24.6	43.3
ViT-Base	Segmenter	-	-	-	-	43.2	43.4	129.6	49.6
	EoMT	94.8	34.1	263.2	40.5	118.4	57.6	171.8	46.5
	TokenMask (Ours)	131.6	42.9	146.3	39.0	150.3	75.6	96.53	46.9
ViT-Large	Segmenter	-	-	-	-	15.0	18.9	400.1	52.3
	EoMT	48.5	13.2	699.9	46.6	64.6	21.7	454.2	54.8
	TokenMask (Ours)	61.9	15.3	501.5	45.1	81.5	24.4	326.3	53.1

mask prediction design and where upsampling is applied. Our experimental analysis follows two axes. First, in the main results we compare the image-space mask head used in EoMT with our token-space formulation. Second, in ablation studies we isolate the role of upsampling by comparing feature-space upsampling, logit-space upsampling applied after mask scoring, and a no-upsampling configuration where predictions are produced at native patch-grid resolution.

4.1. Experimental Setup

Datasets. We evaluate our approach on three widely used panoptic segmentation benchmarks spanning diverse scene types and annotation granularities. ADE20K [19] contains 150 semantic categories across indoor and outdoor environments, and we use its official panoptic training and validation splits. Cityscapes [4] focuses on high-resolution urban street scenes with fine-grained semantic and instance annotations; results are reported on the standard validation set. COCO-Panoptic [12] combines instance-level and semantic annotations over complex natural scenes and serves as a large-scale benchmark for unified panoptic segmentation.

Evaluation Metrics. Computational efficiency is assessed using both theoretical complexity (GFLOPs) and empirical runtime measured in frames per second (FPS). To ensure fair and reproducible runtime evaluation across platforms, we report FPS with batch size one on an NVIDIA A100 [1] using PyTorch FP32 inference as a standardized high-performance reference, and the target embedded platform, an NVIDIA Jetson AGX Orin [2] (64GB LPDDR5, Ampere GPU with 2048 CUDA cores and 64 Tensor Cores) configured in MaxN mode (60W), using TensorRT FP16 inference for deployment analysis.

Segmentation quality is evaluated using standard task-specific metrics: Panoptic Quality (PQ) for panoptic segmentation, mean Average Precision (mAP) for instance segmentation, and mean Intersection-over-Union (mIoU) for semantic segmentation, with all results reported under an embedded deployment setting.

Training Protocol. All models are trained using the AdamW optimizer with identical learning rate schedules and data augmentation strategies across architectural variants. Training duration follows standard practice for each dataset: 160 epochs for ADE20K, 24 epochs for COCO-Panoptic, and 107 epochs for Cityscapes. We use Q=200 queries for ADE20K and COCO, and Q=100 queries for Cityscapes. Models are trained specifically for each target task, rather than reusing a single checkpoint across multiple objectives. All models share identical initialization, decoder depth, query count, and training protocol.

4.2. Main Results

Tables 1, 2 and 3 summarize the accuracy-efficiency tradeoff of TokenMask across multiple datasets and ViT backbones, and compare it to representative segmentation meth-

[1] https://www.nvidia.com/en-us/data-center/a100/

[2] https://developer.nvidia.com/embedded/jetson-orin

Table 2. Performance and efficiency of the proposed TokenMask head on Cityscapes for different ViT backbones. All results are reported at 1024×1024 input resolution.

Backbone	Method	FPS A100 ↑	FPS Orin ↑	GFLOPs ↓	mIoU ↑
ViT-Tiny	Segmenter	32.9	40.1	117.0	73.1
	EoMT	132.0	42.5	45.6	69.7
	TokenMask (Ours)	160.0	53.8	23.2	68.0
ViT-Small	Segmenter	15.7	22.1	285.0	76.6
	EoMT	78.3	22.6	165.4	81.0
	TokenMask (Ours)	104.4	27.2	90.4	79.7
ViT-Base	Segmenter	6.8	9.9	776.0	77.1
	EoMT	37.5	9.6	628.1	81.8
	TokenMask (Ours)	51.4	11.9	356.6	80.4
ViT-Large	Segmenter	20.8	3.4	2248.3	79.1
	EoMT	17.6	3.6	1721.3	83.6
	TokenMask (Ours)	21.9	4.2	1251.6	82.9

Table 3. Performance and efficiency comparison on COCO-Panoptic for instance and panoptic segmentation. All results are reported at 640×640 input resolution. GFLOPs are identical for instance and panoptic settings and therefore reported once.

			Instance			Panoptic		
Backbone	Method	GFLOPs ↓	FPS A100 ↑	FPS Orin ↑	mAP ↑	FPS A100 ↑	FPS Orin ↑	PQ ↑
ViT-Tiny	EoMT	18.39	198.5	148.1	21.5	202.7	145.9	32.8
	TokenMask (Ours)	9.47	218.6	210.8	20.2	213.2	210.9	32.0
ViT-Small	EoMT	64.14	136.2	69.3	29.8	137.4	66.8	41.9
	TokenMask (Ours)	36.66	153.8	106.5	27.5	156.2	107.0	41.0
ViT-Base	EoMT	237.56	107.2	34.5	33.8	107.4	34.8	46.8
	TokenMask (Ours)	144.22	140.4	44.6	31.0	142.6	43.8	44.6
ViT-Large	EoMT	656.43	52.6	13.3	40.9	52.6	12.4	53.3
	TokenMask (Ours)	497.92	64.9	15.0	37.4	64.9	15.0	50.9

ods such as Mask2Former, Segmenter, and lightweight architectures like EoMT and LiPS.

Across datasets, tasks, and backbone scales, TokenMask consistently reduces computational cost while maintaining competitive segmentation accuracy. On ADE20K panoptic segmentation (Table 1), this translates into 53.6

Across all datasets, tasks, and backbone scales, TokenMask systematically consistently reduces computational cost while and improves runtime performance while maintaining competitive segmentation accuracy. On ADE20K panoptic segmentation (Table 1), compared to EoMT, this translates into 53.6%, 48.0%, and 44.4% GFLOPs reductions for ViT-Tiny/Small/Base, respectively, with corresponding speedups of up to +53.7% on Nvidia Jeston Orin. For ViT-Large, computation is reduced by 28.3% with corresponding runtime gains of +15.9% (Orin). These efficiency gains come with only moderate PQ variations (typically within 1.5-2.8 points), indicating that a large fraction of feature-space computation can be removed with limited impact on segmentation quality. Relative to Mask2Former and heavier Segmenter variants, TokenMask reduces computation by up to 75–90% and delivers up to 2× inference speedup. Against lightweight architectures such as LiPS and Segmenter (Tiny/Small), TokenMask improves efficiency, reducing computation by 20–60% and delivering up to 30–45% inference speedups.

A similar trend is observed for semantic segmentation on ADE20K (Table 1) compared to EoMT: on ViT-Small, GFLOPs are reduced by 47% with a +53.3% speedup and only a minor mIoU drop, while on ViT-Base, TokenMask reduces computation and improves runtime while matching or slightly improving mIoU.

These computational reductions translate directly into lower activation memory. As reported in Table 4, peak allocated memory decreases consistently across scales (e.g., 302.2→189.6 MB for ViT-Small and 763.6→499.2 MB for ViT-Base). For larger backbones, the absolute savings become substantial (e.g., over 380 MB reduction for ViT-

Table 4. Peak GPU memory during forward profiling (MB). "Peak alloc." denotes maximum allocated CUDA memory, and "Peak res." the maximum memory reserved by the CUDA caching allocator. Measurements use batch size 1, input resolution 640^2, `torch.compile=max-autotune`, and AMP FP16.

	EoMT		TokenMask (Ours)	
Backbone	**Peak alloc.**	**Peak res.**	**Peak alloc.**	**Peak res.**
ViT-Tiny	137.3	250.0	98.2	192.0
ViT-Small	302.2	488.0	189.6	228.0
ViT-Base	763.6	1168.0	499.2	682.0
ViT-Large	1746.6	2210.0	1363.0	1446.0
ViT-Giant	5277.0	5908.0	4667.8	4820.0

Large).

On COCO-Panoptic (Table 3), TokenMask exhibits similar trends for both instance and panoptic tasks. For ViT-Tiny and ViT-Small, compute is nearly halved (18.39→9.47 and 64.14→36.66 GFLOPs) and throughput increases consistently. For instance segmentation, inference speed on Orin increases from 148.1 to 210.8 FPS (+42.3%) for ViT-Tiny and from 69.3 to 106.5 FPS (+53.7%) for ViT-Small. For panoptic segmentation, Orin FPS improves from 145.9→210.9 (+44.5%) and 66.8→107.0 (+60.2%). Accuracy remains close to the baseline across scales (e.g., PQ 32.8→32.0 and 41.9→41.0), indicating that the reformulation primarily restructures computation rather than substantially altering recognition capacity.

Under higher-resolution inputs, Cityscapes confirms the same scaling behavior (Table 2). Compared to EoMT, for ViT-Base, GFLOPs decrease from 628.06 to 356.62 (-43.2%), with FPS improving from 9.6→11.9 on Orin (+24.0%), while mIoU remains competitive (81.8→80.4). Similar gains are observed for ViT-Small (22.6→27.2 FPS on Orin) with limited accuracy change (81.0→79.7). In contrast to Segmenter, TokenMask reduces computation by more than 2× across backbones (e.g., 776.0→356.6 GFLOPs on ViT-Base), while improving FPS and maintaining comparable or better mIoU.

Qualitative comparisons in Figure 3 further support these findings: across diverse scene layouts and object scales, TokenMask produces segmentation results that are visually consistent with the ground truth. Both small instances and large structural regions are accurately delineated, indicating that the reduced computation does not compromise spatial coherence or instance separation.

4.3. Ablation Studies

We conduct a structured ablation study to isolate the impact of individual design choices within the mask prediction pipeline. Specifically, we study: **(1)** relocating interpolation from feature space to logit space; **(2)** removing upsampling entirely; **(3)** varying output stride as a deployment-level parameter; and **(4)** numerical consistency under mixed-precision TensorRT inference. Unless stated otherwise, all ablations use ADE20K (panoptic), 640×640 inputs, on NVIDIA A100 GPUs.

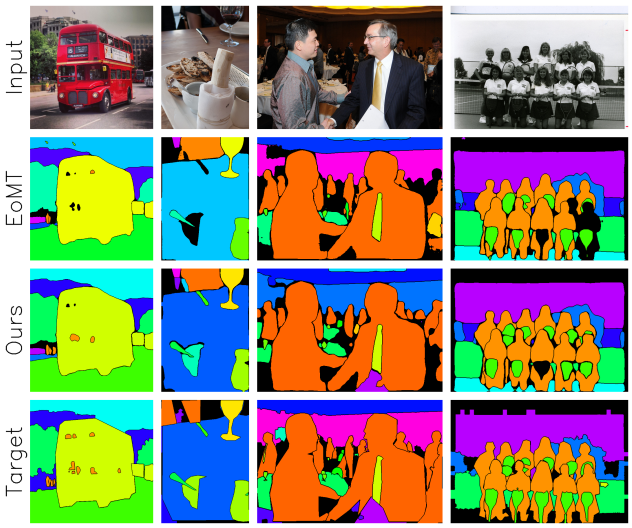

Figure 3. Qualitative comparison on panoptic segmentation.

Relocating Interpolation: Feature vs Logit Space. We first retain the classical feature-map mask head and modify only the location of spatial interpolation. In the baseline EoMT formulation, interpolation is applied to C-channel feature maps prior to mask scoring. We instead apply interpolation to the Q-channel mask logits after scoring, while keeping all other components unchanged. As shown in Table 5, switching from feature-space to logit-space upsampling leads to a consistent but moderate PQ decrease (between -1.1 and -2.2 points) across all backbones. However, this accuracy drop remains limited compared to the substantial reductions in computational cost and the improvement in inference speed. This corresponds to FLOPs reductions of 53.6%, 48.0%, 44.4%, and 28.3% for ViT-Tiny, ViT-Small, ViT-Base, and ViT-Large, respectively. These results indicate that most of the spatial refinement provided by feature-space interpolation is redundant.

Removing Upsampling Entirely. We next evaluate the extreme configuration where no spatial upsampling is applied and masks are produced directly at patch-grid resolution. This removes both feature-space and logit-space interpolation. The purpose of this experiment is to assess whether explicit spatial refinement is fundamentally required for competitive segmentation performance. Results in Tables 5 and 6 show that completely removing the upsampling operation yields only marginal additional

Table 5. Ablation of upsampling strategies applied to the EoMT.

Method	Backbone	Upsampling	FPS ↑	GFLOPs ↓	PQ ↑
EoMT	ViT-Tiny	Feature	189.0	20.85	29.5
		Logit	215.7	9.68	28.4
		None	217.0	9.58	28.1
	ViT-Small	Feature	129.3	71.69	38.0
		Logit	153.7	37.28	35.8
		None	154.5	37.18	35.7
	ViT-Base	Feature	94.8	263.15	40.5
		Logit	135.3	146.33	39.0
		None	135.6	146.23	38.9
	ViT-Large	Feature	48.5	699.89	46.6
		Logit	61.4	501.51	45.4
		None	62.3	501.41	45.0

Table 6. Ablation study on the role of logit-space upsampling in the proposed TokenMask head.

Method	Backbone	Upsampling	FPS ↑	GFLOPs ↓	PQ ↑
TokenMask (Ours)	ViT-Tiny	Logit	202.9	9.68	27.7
		None	212.8	9.58	28.1
	ViT-Small	Logit	150.8	37.28	35.2
		None	154.3	37.18	35.5
	ViT-Base	Logit	131.6	146.33	39.0
		None	136.4	146.23	38.0
	ViT-Large	Logit	61.9	501.51	45.1
		None	61.9	501.41	45.1

Table 7. Effect of output stride for TokenMask with a ViT-Small.

Output Stride	FPS ↑	GFLOPs ↓	PQ ↑
1	154.0	38.73	33.9
4	196.0	37.19	35.1
8	199.7	37.12	33.9
16	197.8	37.09	34.5

computational savings compared to logit-space interpolation (around 0.1 GFLOPs across all backbones), corresponding to less than 1% relative reduction. This indicates that most of the computational overhead is already eliminated when shifting interpolation to logit space. We also observe a small but consistent PQ decrease (between -0.1 and -0.4 points), which, combined with the negligible additional computational savings, suggests that logit-space interpolation is nearly cost-free yet still beneficial for maintaining segmentation quality.

Output Stride as a Deployment Knob. We further analyze the effect of varying the mask output stride when using logit-space upsampling. Instead of fixing the output resolution, we evaluate multiple stride values to study the trade-off between spatial resolution and computational cost. As shown in Table 7, stride 4 emerges as the best trade-off, achieving the highest PQ (35.1) while substantially improving inference speed compared to stride 1 (+27% FPS) and reducing computational cost. Although larger strides (8 and 16) provide marginal additional FLOPs reductions (below 0.3%) and slight throughput gains, they consistently degrade PQ (up to -1.2 points).

5. Conclusions

We introduced TokenMask, a token-space mask head that reformulates query-based segmentation by computing mask logits directly from query-token affinities, thereby eliminating explicit image-space feature reconstruction. By aligning mask prediction with the intrinsic token representation of ViTs and leveraging GEMM-friendly operations, TokenMask provides a hardware-conscious and structurally streamlined approach to semantic, instance, and panoptic segmentation. Our analysis demonstrated that relocating interpolation from feature space to logit space substantially improves efficiency, and that even removing upsampling entirely remains viable in many regimes. These findings clarify the role of upsampling in query-based segmentation and expose favorable scaling behavior with respect to embedding dimension and spatial resolution.

Across all evaluated datasets and over multiple ViT backbone scales, TokenMask consistently reduces computational complexity, increases throughput, and lowers peak memory usage while maintaining competitive segmentation accuracy. Compared to EoMT, TokenMask reduces computational cost by up to 53.6%, improves runtime by up to 1.5×, and decreases peak memory usage by several hundreds of megabytes across backbone scales, while incurring only minor accuracy variations (typically within 1.5-2.8 PQ points).

Overall, our results suggest that efficient segmentation with ViTs does not require explicit spatial reconstruction, opening new perspectives for lightweight, deployment-oriented design in embedded vision systems.

Acknowledgments

This work was partially funded by the French Defence Innovation Agency (Agence de l'innovation de défense, AID).

References

[1] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *Proc. of the Eur. Conf. on Computer Vision*, pages 213–229, 2020. 1, 2

[2] Bowen Cheng, Alex Schwing, and Alexander Kirillov. Per-pixel classification is not all you need for semantic segmentation. *Advances in Neural Information Processing Systems*, 34:17864–17875, 2021. 1, 2

[3] Bowen Cheng, Ishan Misra, Alexander G Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *Proc. of the IEEE/CVF Conf. on Computer Vision and Pattern Recognition*, pages 1290–1299, 2022. 1, 2

[4] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. *IEEE Conf. on Computer Vision and Pattern Recognition*, pages 3213–3223, 2016. 5

[5] Bo Dong, Pichao Wang, and Fan Wang. Head-free lightweight semantic segmentation with linear transformer. In *Proc. of the AAAI conf. on artificial intelligence*, pages 516–524, 2023. 5

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *Int. Conf. Learn. Represent.*, 2021. 1

[7] Calvin Galagain, Martyna Poreba, and François Goulette. Is semantic slam ready for embedded systems? a comparative survey. *arXiv preprint arXiv:2505.12384*, 2025. 1

[8] Calvin Galagain, Martyna Poreba, François Goulette, and Cyrill Stachniss. Lips: Lightweight panoptic segmentation for resource-constrained robotics, 2026. 2

[9] Jitesh Jain, Jiachen Li, Mang Tik Chiu, Ali Hassani, Nikita Orlov, and Humphrey Shi. Oneformer: One transformer to rule universal image segmentation. In *Proc. of the IEEE/CVF Conf. on Computer Vision and Pattern Recognition*, pages 2989–2998, 2023. 2

[10] Tommie Kerssies, Niccolo Cavagnero, Alexander Hermans, Narges Norouzi, Giuseppe Averta, Bastian Leibe, Gijs Dubbelman, and Daan de Geus. Your vit is secretly an image segmentation model. In *Proc. of the Computer Vision and Pattern Recognition*, pages 25303–25313, 2025. 1, 2

[11] Fangjian Lin, Zhanhao Liang, Sitong Wu, Junjun He, Kai Chen, and Shengwei Tian. Structtoken: Rethinking semantic segmentation with structural prior. *IEEE Transactions on Circuits and Systems for Video Technology*, 33(10):5655–5663, 2023. 2

[12] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Eur. Conf. on Computer Vision*, pages 740–755. Springer, 2014. 5

[13] Robin Strudel, Ricardo Garcia, Ivan Laptev, and Cordelia Schmid. Segmenter: Transformer for semantic segmentation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 7262–7272, 2021. 2

[14] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers, 2021. 2

[15] Bowen Zhang, Zhi Tian, Quan Tang, Xiangxiang Chu, Xiaolin Wei, Chunhua Shen, and Yifan Liu. Segvit: Semantic segmentation with plain vision transformers, 2022. 2

[16] Bowen Zhang, Liyang Liu, Minh Hieu Phan, Zhi Tian, Chunhua Shen, and Yifan Liu. Segvitv2: Exploring efficient and continual semantic segmentation with plain vision transformers, 2023. 2

[17] Li Zhang, Jiachen Lu, Sixiao Zheng, Xinxuan Zhao, Xiatian Zhu, Yanwei Fu, Tao Xiang, Jianfeng Feng, and Philip H. S. Torr. Vision transformers: From semantic segmentation to dense prediction. *International Journal of Computer Vision*, 132(12):6142–6162, 2024. 2

[18] Sixiao Zheng, Jiachen Lu, Hengshuang Zhao, Xiatian Zhu, Zekun Luo, Yabiao Wang, Yanwei Fu, Jianfeng Feng, Tao Xiang, Philip H.S. Torr, and Li Zhang. Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers. In *CVPR*, 2021. 2

[19] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition*, pages 633–641, 2017. 5